

Gradient Boosting

What is the model?

Ensemble method that builds trees sequentially

Each new tree corrects errors of previous trees

What does it consist of?

- Weak learners (usually shallow decision trees)
- Sequential additive model
- Gradient descent to minimize loss function

Gradient Boosting Details

What is it used for?

- Regression and classification tasks
- Works well on complex, non-linear data

How does it update weights?

$$W_{\text{new}} = W - \eta \cdot g$$

Where:

$$g = \frac{\partial \text{Loss}}{\partial W}$$

(gradient)

η = learning rate

Intuition:

- Gradient shows how wrong the current prediction is
- Update moves prediction slightly toward the correct value

When to use it?

- Datasets with complex patterns
- When high accuracy is required
- Can handle both regression & classification



XGBoost (Extreme Gradient Boosting)

What is the model?

- Gradient boosting framework optimized for speed and performance
- Uses regularization to prevent overfitting

What does it consist of?

- Weak learners (decision trees)
- Additive trees built sequentially
- Regularization terms for tree complexity

XGBoost Details

What is it used for?

- Regression and classification
- High-performance datasets, competitions, and large data

How does it update weights?

Leaf weight formula:

$$W = -\frac{\sum g_i}{\sum h_i + \lambda}$$

Where:

g_i = gradient

h_i = second derivative (Hessian)

λ = L2 regularization

Update step with learning rate e

$$W_{\{new\}} = W - learning\ rate \cdot W = -\frac{\sum g_i}{\sum h_i + \lambda}$$

Intuition:

- Adjusts leaves to minimize loss
- Regularization prevents overfitting

When to use it?

- Large datasets
- When high accuracy and speed are required
- Competitions (Kaggle, etc.)

LightGBM (Light Gradient Boosting Machine)

What is the model?

- Gradient boosting framework optimized for speed and memory efficiency
- Builds trees leaf-wise for better accuracy

What does it consist of?

- Weak learners (decision trees)
- Leaf-wise growth strategy
- Gradient-based one-side sampling and exclusive feature bundling



LightGBM Details

What is it used for?

- Regression and classification
- Large datasets with high-dimensional features

How does it update weights?

- Uses same gradient boosting formula:
- $W_{\{new\}} = W - lr \cdot g$
- Gradient computed from loss function
- Leaf-wise growth chooses leaves with largest loss reduction

When to use it?

- Large-scale datasets
- When training speed and memory efficiency are important
- When higher accuracy than depth-wise trees is desired

CatBoost (Categorical Boosting)

What is the model?

- Gradient boosting algorithm developed by Yandex
- Handles categorical features natively without one-hot encoding
- Reduces prediction shift using ordered boosting

What does it consist of?

- Weak learners (decision trees)
- Gradient boosting framework
- Special categorical feature handling
- Ordered boosting to prevent overfitting



CatBoost Details

What is it used for?

- Regression and classification tasks
- Datasets with many categorical features
- High-performance tasks requiring robust generalization

How does it update weights?

$$g = \frac{\partial Loss}{\partial W}$$

- Uses gradient boosting formula:
- $W_{\{new\}} = W - \eta \cdot g$
- Where:
- (gradient)
- η = learning rate

Intuition:

- Each new tree focuses on errors from previous trees
- Ordered boosting ensures less overfitting on small datasets

When to use it?

- When the dataset has categorical variables
- Regression or classification tasks with heterogeneous data
- When you want a robust, high-accuracy model

CatBoost & Categorical Features (Simple)

No One-Hot Encoding Needed

CatBoost can handle categories like “City” or “Country” directly.

No need to convert them into many 0s and 1s (one-hot).

How It Works (Simply)

Converts each category into a number based on target values (like the average).

Makes sure the number doesn’t “peek” at its own target → avoids overfitting.

Example

Feature1: City, F2: Price

City	Price	Encoded
Cairo	100	150
Giza	150	150
Cairo	200	40
Alexandria	120	120

These numbers go directly into the tree.

Key Advantage

- Works well for high-cardinality features
- Saves memory and prevents overfitting